

1. import heapq

```
def a_star_search(graph, heuristics, start, goal):
    # Priority queue to store (cost + heuristic, cost, current_node, path)
    open_list = [(0 + heuristics[start], 0, start, [start])]
    visited = set()

    while open_list:
        _, current_cost, current_node, path = heapq.heappop(open_list)

        if current_node == goal:
            return path, current_cost

        if current_node in visited:
            continue
        visited.add(current_node)

        for neighbor, distance in graph[current_node].items():
            if neighbor not in visited:
                total_cost = current_cost + distance
                f_cost = total_cost + heuristics[neighbor]
                heapq.heappush(open_list, (f_cost, total_cost, neighbor, path + [neighbor]))

    return None

# Simplified Romania Map data
graph = {
    'Arad': {'Sibiu': 140, 'Timisoara': 118, 'Zerind': 75},
    'Sibiu': {'Arad': 140, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
    'Rimnicu Vilcea': {'Sibiu': 80, 'Pitesti': 97, 'Craiova': 146},
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Pitesti': {'Rimnicu Vilcea': 97, 'Bucharest': 101, 'Craiova': 138},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101}
}
heuristics = {'Arad': 366, 'Sibiu': 253, 'Rimnicu Vilcea': 193, 'Fagaras': 176, 'Pitesti': 100,
              'Bucharest': 0, 'Timisoara': 329, 'Zerind': 374, 'Craiova': 160}

path, cost = a_star_search(graph, heuristics, 'Arad', 'Bucharest')
print(f"Optimal Path: {path}")
print(f"Total Distance: {cost}km")
```

2. import random

```
def vacuum_cleaner(n):
    # Create an n x n grid with random 0s (clean) and 1s (dirty)
    room = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

    print("Initial Room State:")
    for row in room:
        print(row)

    rounds = 0
    # The vacuum goes row by row, column by column
    for i in range(n):
        for j in range(n):
            rounds += 1
            print(f"\nVacuum is at position ({i}, {j})")
```

```

        if room[i][j] == 1:
            print("Status: Dirty. Cleaning it now...")
            room[i][j] = 0
        else:
            print("Status: Already clean.")

    print(f"\nAll rooms cleaned in {rounds} rounds.")
    print("Final Room State:")
    for row in room:
        print(row)

# Run for a 4x4 room
vacuum_cleaner(4)

```

3. import math

```

def minimax(depth, node_index, is_maximizing_player, scores, target_depth):
    # Base case: if we reached the leaf node
    if depth == target_depth:
        return scores[node_index]

    if is_maximizing_player:
        best_val = -math.inf
        # Check left and right children
        for i in range(2):
            val = minimax(depth + 1, node_index * 2 + i, False, scores, target_depth)
            best_val = max(best_val, val)
        return best_val
    else: [cite: 66]
        best_val = math.inf
        for i in range(2):
            val = minimax(depth + 1, node_index * 2 + i, True, scores, target_depth)
            best_val = min(best_val, val)
        return best_val

# Leaf nodes scores
scores = [3, 5, 2, 9, 12, 5, 23, 23]
tree_depth = int(math.log2(len(scores)))

optimal_value = minimax(0, 0, True, scores, tree_depth)
print(f"The optimal value is: {optimal_value}")

```

4. INFINITY = float('inf')

```

def alpha_beta(node, depth, alpha, beta, maximizing):
    # Base case
    if depth == 0 or node.get('terminal', False):
        return node.get('value', 0)

    if maximizing:
        max_val = -INFINITY
        for child in node.get('children', []):
            val = alpha_beta(child, depth-1, alpha, beta, False)
            max_val = max(max_val, val)
            alpha = max(alpha, val)
            if beta <= alpha: # Prune

```

```

        break
    return max_val
else:
    min_val = INFINITY
    for child in node.get('children', []):
        val = alpha_beta(child, depth-1, alpha, beta, True)
        min_val = min(min_val, val)
        beta = min(beta, val)
        if beta <= alpha: # Prune
            break
    return min_val

```

Example tree

```

tree = {
    'children': [
        {'value': 3, 'terminal': True},
        {'value': 5, 'terminal': True},
        {'children': [
            {'value': 6, 'terminal': True},
            {'value': 9, 'terminal': True},
        ]},
        {'value': 1, 'terminal': True},
    ]
}

```

Run

```

result = alpha_beta(tree, 3, -INFINITY, INFINITY, True)
print(result)

```

5. import random

```

board = [' '] * 9
turn = 'X'

```

```

def draw():
    for i in range(0, 9, 3):
        print(board[i], '|', board[i+1], '|', board[i+2])
        if i < 6: print('---+---+---')
    print()

```

```

def check_win():
    wins = [(0,1,2),(3,4,5),(6,7,8),
            (0,3,6),(1,4,7),(2,5,8),
            (0,4,8),(2,4,6)]
    for a,b,c in wins:
        if board[a] == board[b] == board[c] != ' ':
            return board[a]
    return None

```

```

while True:
    draw()

    if turn == 'X':
        try:
            pos = int(input("Enter 1-9: ")) - 1
            if board[pos] != ' ': continue
        except:
            continue
    else:

```

```
pos = random.choice([i for i in range(9) if board[i] == ' '])  
print("Machine played:", pos+1)
```

```
board[pos] = turn  
win = check_win()
```

```
if win or ' ' not in board:  
    draw()  
    print(win, "wins!" if win else "Draw!")  
    break
```

```
turn = 'O' if turn == 'X' else 'X'
```